

LINQ Min Method in C# with Examples

In this article, I will discuss the **LINQ Min Method in C#** with examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Max Aggregate Method in C#](#) with Examples. The LINQ Min Method belongs to the category of Aggregate Operators.

What is the LINQ Min Method in C#?

The LINQ Min method finds the smallest numeric value in a collection on which it is applied. It's part of the **System.Linq** namespace can be used with arrays, lists, or any type that implements the **IEnumerable<T>** interface. The Min method comes in various forms, allowing you to find the minimum value in different ways depending on the type of the collection and the data it contains.

Example to Understand Min Method in C#:

Let us understand the LINQ Min() method with Examples in C# using Method and Query Syntax. The following example returns the smallest number from the collection using query and method syntax. The point that you need to remember is that the Min Method will work with numeric values only. The collection stores integer values, so we can apply the Min Method. We don't have any operator called min in the Query Syntax. So here, we need to use mixed syntax.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,
            //Using Method Syntax
            int MSLowestNumber = intNumbers.Min();
            //Using Query Syntax
            int QSLowestNumber = (from num in intNumbers
                                select num).Min();
            Console.WriteLine("Lowest Number = " + MSLowestNumber);
            Console.ReadKey();
        }
    }
}
```

Output: **Lowest Number = 10**

Example to Understand LINQ Min Method with Where Extension Method using C#

Let us see an example to Understand how we can use the LINQ Min Method and the Where Extension Method in C# using both Method and Query Syntax. Now, our requirement is to return the smallest number from the collection where the number is greater than 50. The following example code exactly does the same.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,
            //Using Method Syntax
            int MSLowestNumber = intNumbers.Where(num => num > 50).Min();
            //Using Query Syntax
            int QSLowestNumber = (from num in intNumbers
                                where num > 50
                                select num).Min();
            Console.WriteLine("Lowest Number = " + MSLowestNumber);
            Console.ReadKey();
        }
    }
}

```

Output: **Lowest Number = 60**

Example to Understand How to use LINQ Min Method with Predicate in C#

Let us see an example to Understand How to use the LINQ Min Method with Predicate in C# using both Method and Query Syntax. Instead of using the Where Extension method to filter the data, we can also use the other overloaded version of the Min Extension method, which takes a Predicate as a parameter, and using the predicate, we can write the logic to filter the data.

In the below example, within the Min Method, we use a Predicate and provide the condition of whether the number is greater than 50. If the number is greater than 50, then we are returning that number; else, we are returning 0. The following example will return the smallest number greater than 50.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,
            //Using Method Syntax
            int MSLowestNumber = intNumbers.Where(num => num > 50).Min(num =>
                if (num > 50)
                    return num;
                else
                    return 0;
            ));
        }
    }
}

```

```

        Console.WriteLine("Lowest Number = " + MSLowestNumber);
        Console.ReadKey();
    }
}
}

```

Output: **Lowest Number = 60**

Example to Understand LINQ Min Method with Complex Type in C#:

Let us see an example of using the LINQ Min Method with Complex Data Type in C# using both Method and Query Syntax. We are going to work with the following Employee class. As you can see, it is a very simple class with four properties: **ID**, **Name**, **Salary**, and **Department**. Here, we also created one method, i.e., **GetAllEmployees()**, which will return the list of all the employees, which will be our data source.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            List<Employee> listStudents = new List<Employee>()
            {
                new Employee{ID= 101,Name = "Preety", Salary = 10000, Department = "Sales"},
                new Employee{ID= 102,Name = "Priyanka", Salary = 15000, Department = "Sales"},
                new Employee{ID= 103,Name = "James", Salary = 50000, Department = "Sales"},
                new Employee{ID= 104,Name = "Hina", Salary = 20000, Department = "IT"},
                new Employee{ID= 105,Name = "Anurag", Salary = 30000, Department = "Sales"},
                new Employee{ID= 106,Name = "Sara", Salary = 25000, Department = "IT"},
                new Employee{ID= 107,Name = "Pranaya", Salary = 35000, Department = "Sales"},
                new Employee{ID= 108,Name = "Manoj", Salary = 11000, Department = "Sales"},
                new Employee{ID= 109,Name = "Sam", Salary = 45000, Department = "Sales"},
                new Employee{ID= 110,Name = "Saurav", Salary = 25000, Department = "Sales"}
            };
            return listStudents;
        }
    }
}

```

Now, our requirement is to return the lowest salary among all the employees. The following example returns the lowest salary among all the employees using Method and Query Syntax. Here, we specify the numeric Salary column using a lambda expression to the Min method.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSLowestSalary = Employee.GetAllEmployees()
                .Min(emp => emp.Salary);

            //Using Query Syntax
            var QSLowestSalary = (from emp in Employee.GetAllEmployees()
                select emp).Min(e => e.Salary);

            Console.WriteLine("Lowest Salary = " + MSLowestSalary);
            Console.ReadKey();
        }
    }
}
```

Output: **Lowest Salary = 10000**

Example using Min and Where Extension Method in C#:

Let us see an example of using both LINQ Min and Where Extension Methods with Complex Type using Method and Query Syntax. Our requirement is to return the Lowest Salary in the IT department. The following example exactly does the same. Using the Where Extension Method, we are filtering the IT department employees, and using the Min method, we are specifying the Salary numeric column, which will return the Lowest Salary of the IT department.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSLowestSalary = Employee.GetAllEmployees()
                .Where(emp => emp.Department == "IT")
                .Min(emp => emp.Salary);

            //Using Query Syntax
            var QSLowestSalary = (from emp in Employee.GetAllEmployees()
                where emp.Department == "IT"
                select emp).Min(e => e.Salary);
        }
    }
}
```

```

        Console.WriteLine("IT Department Lowest Salary = " + QSLowestSalary);
        Console.ReadKey();
    }
}

```

Output: IT Department Lowest Salary = 10000

Let's rewrite the previous example using the custom predicate. As you can see, within the Min method, we are checking the Department property value, and if the Department value is It, we are returning the corresponding Salary. Else, we are returning null.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax and Predicate
            var MinumumSalaryMS = Employee.GetAllEmployees()
                .Min(emp => {
                    if (emp.Department == "IT")
                        return emp.Salary;
                    else
                        return null;
                });
            Console.WriteLine("IT Department Lowest Salary = " + MinumumSalaryMS);
            Console.ReadKey();
        }
    }
}

```

Output: IT Department Lowest Salary = 10000

Handling Nullable Types

The Min method also has overloads for handling nullable numeric types (int?, double?, etc.). The Min method will ignore null values. For a better understanding, please have a look at the below example.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int?[] nullableNumbers = { 10, 1, 2, null, 4, 5 };

```

```

        int? min = nullableNumbers.Min();
        Console.WriteLine("Min = " + min);
        Console.ReadKey();
    }
}

```

Output: Min = 1

When should you use the LINQ Min Method in C#?

The Min method in LINQ is used when finding the minimum value in a collection. It can be used in various scenarios, such as:

- **Finding the Lowest Value:** When you have a collection of numeric types (like int, float, decimal) and you want to find the smallest number.
- **Minimum of Custom Objects:** If you have a collection of custom objects, you can use Min with a lambda expression to find the object with the lowest value of a specific property. For example, finding the employee with the lowest salary in a list of employees.
- **Minimum with Nullable Types:** LINQ's Min method can handle nullable types (int?, float?, etc.), and it will return null if the collection is empty or consists only of null values.
- **Querying Databases:** When using LINQ to Entities or LINQ to SQL to query databases, the Min method can be used to retrieve the minimum value from a column in a database table.
- **Aggregation in Complex Queries:** In complex LINQ queries, Min can be an essential part of aggregation operations, especially after grouping elements using GroupBy or similar operations.
- **Comparing Custom Data Types:** For custom data types, implementing IComparable allows you to use Min to find the 'smallest' object according to the comparison logic defined in your type.
- **Time-based Data:** In collections containing date and time data, Min is useful for finding the earliest date.

In the next article, I will discuss the [LINQ Average Method in C#](#) with Examples. I hope you understand the need and use of the [Linq Min Aggregate Method in C#](#) with Examples. I hope you enjoy this article.